Examples

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

What this library does 1
One object: recipe = live dialogue = rollout driver 2
Index 2

Worked examples for the multi-model conversation + interpretability harness, ordered simple → advanced. Every snippet assumes you import the package as a library:

```
from interlens import Conversation, AutoModelParticipant, ModelParticipant
    # etc.
```

For the per-symbol API reference (one page per class and function, with signatures and source links), see `docs/reference/`.

Install the package first (`pip install interlens`, or `pip install -e .` from the library root for development), then run any snippet as a normal script — e.g. `python your_script.py`. GPU examples need CUDA; a small model (`Qwen/Qwen2.5-0.5B-Instruct`) runs on CPU/MPS for smoke tests.

What this library does

Orchestrates turn-taking between two (or more) **participants** — local HF models or hosted-API models — over a shared, perspective-neutral **transcript**, with first-class **interpretability** (activation capture, steering, activation patching, token logprobs) hooked into the *same* generation path as real turns. It scales from one interactive conversation to thousands of checkpointed, multi-GPU **rollouts**.

One object: recipe = live dialogue = rollout driver

A `Conversation` (with lazy `Participant`s) is at once the serializable **recipe**, the **live dialogue** you drive in-process, and the **rollout driver** you expand over data/N samples. There are no separate template/spec/config types — build it up functionally (`.turns(6).data(ds).analyzer(grade)`), run it, or `.rollout()` it (01, 02, 08). Persist it with `save`/`load` (recipe + transcript on disk, resumable — 04).

Index

#	File	Covers
01	Quickstart	Two models talk in ~5 lines; read the transcript
02	Conversations	Manual builds, private/shared framing, moderator, turn-taking, stop conditions, `branch`, ephemeral `sample`, reasoning visibility
03	Participants & models	Model resolution (config.model_type, auto-derived flags), `ModelParticipant` knobs, `kv_reuse`, `APIParticipant`, mixed local+API
04	Context & serialization	Context policies, the conversation-as-recipe, `save`/`load`, resume
05	Tools	Define a `Tool`, register it, the tool-calling loop
06	Hooks	`MessageHook` approve / deny / edit (the LLM-judge seam)

#	File	Covers
07	Interpretability	Capture, steering, ablation, patching, logprobs
08	Rollouts & scale	`conv.rollout`, data-driven rollouts (`dataset_field`), `interlens.run` (multi-lineup), `TokenBudget` (matched compute), multi-GPU, batched co-stepping, `analyzer`, checkpoint/resume
09	Advanced interp pipelines	Causal tracing (capture → patch across branches), steering sweeps, probe-in-the-loop `analyze`
10	Arena	Scoreable multi-agent evaluations: scenarios with exact solvers/scorers, episode drivers, budgets + spend metering, communication styles, the Inspect integration
11	Scorable negotiation & oracles	Formal bargaining games, rational policy baselines, equilibrium/best-response/ acceptance oracles, regret and divergence analysis
12	Control Tower	Use an Interlens local model as a Control Tower untrusted policy while preserving its monitorable tool boundary

Related: the pipeline **performance** profiler lives at `tests/profile_pipeline.py`; the family self-registry lives on `ModelParticipant` (`MODEL_TYPES` + `for_model_type`), and model loading / chat-flag derivation in `load.py`.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.